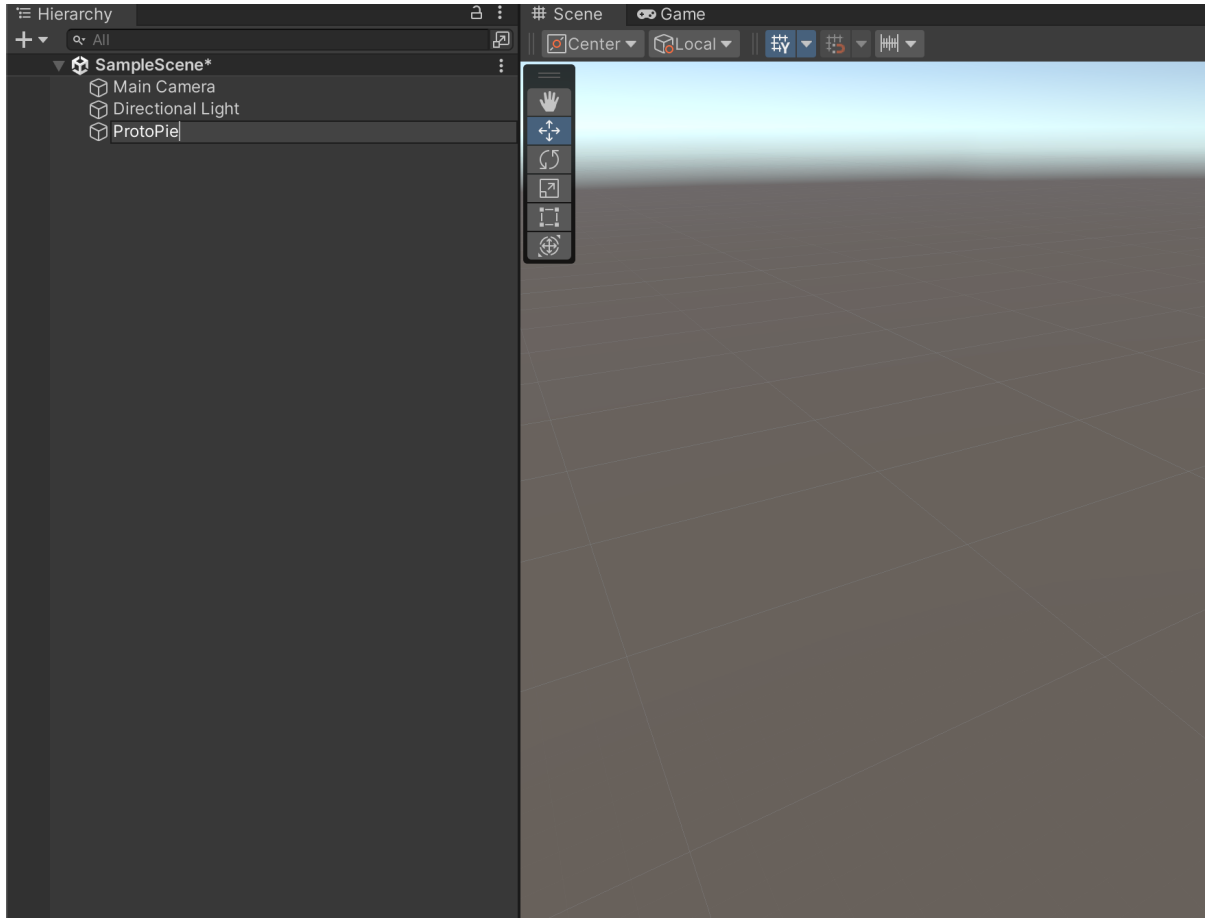
Instructions for ProtoPieUnity Package

Freddie Lee

Emerging Technology
ProtoPie

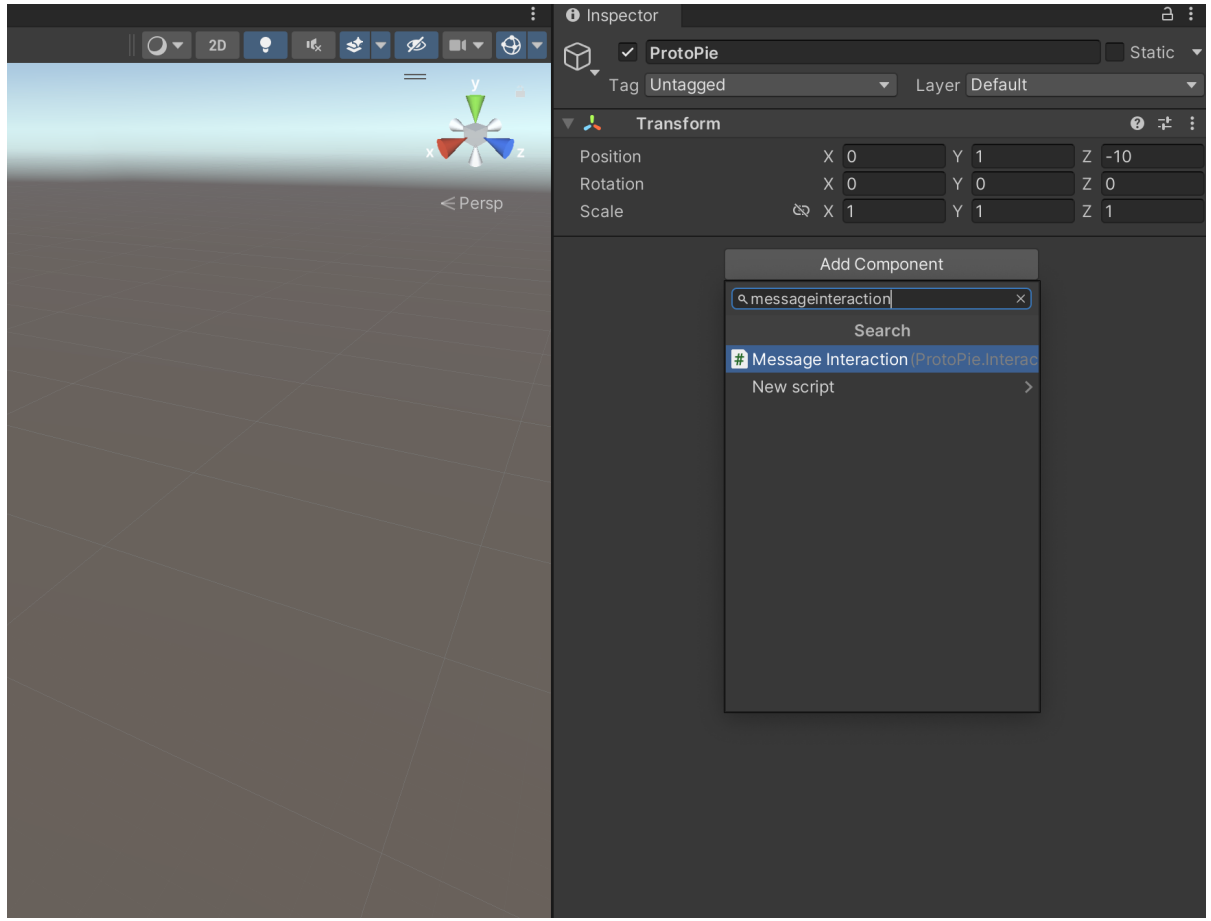
April 2024

1. Basic settings



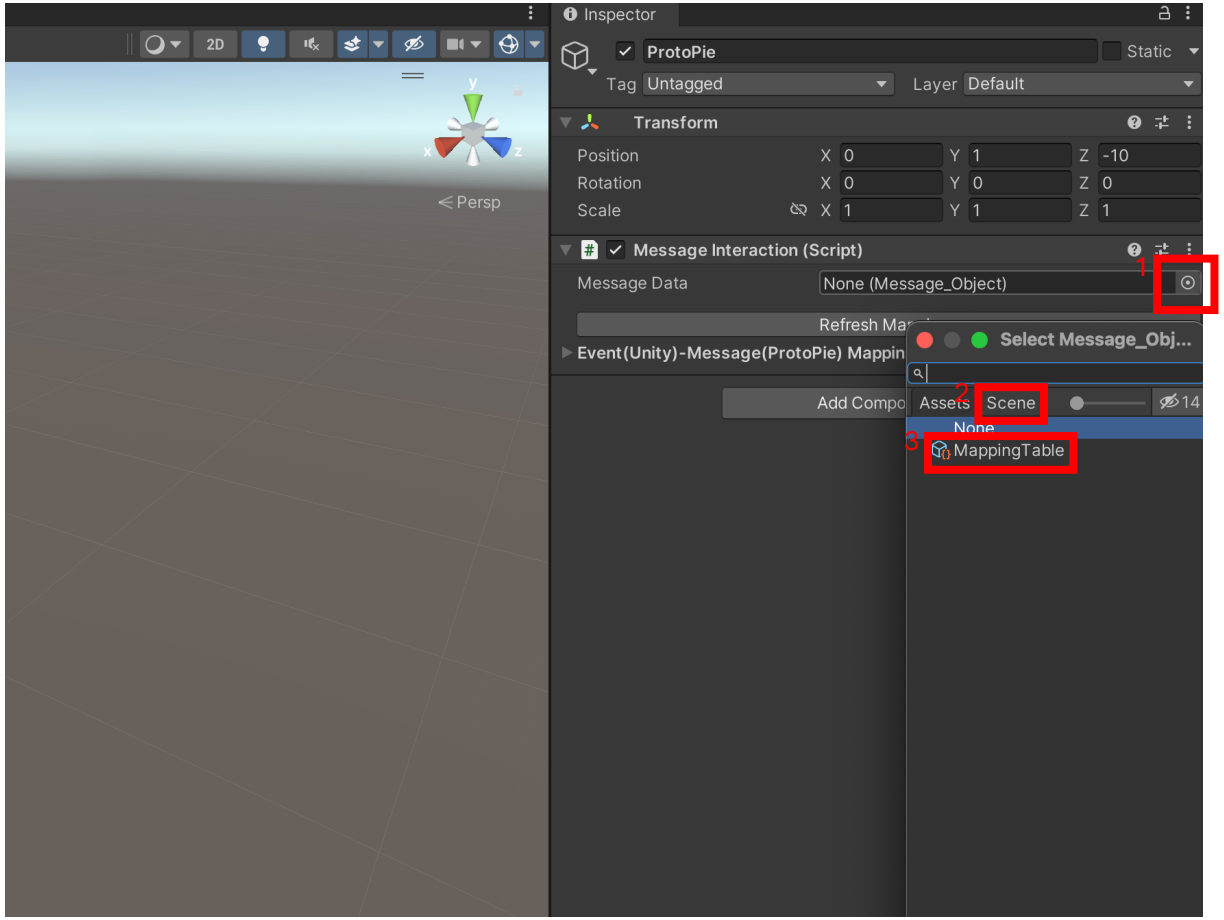
1. Create an empty object and name it ProtoPie

The name must be “ProtoPie” (case sensitive) because ProtoPie Connect detects the object for message interactions using that name.



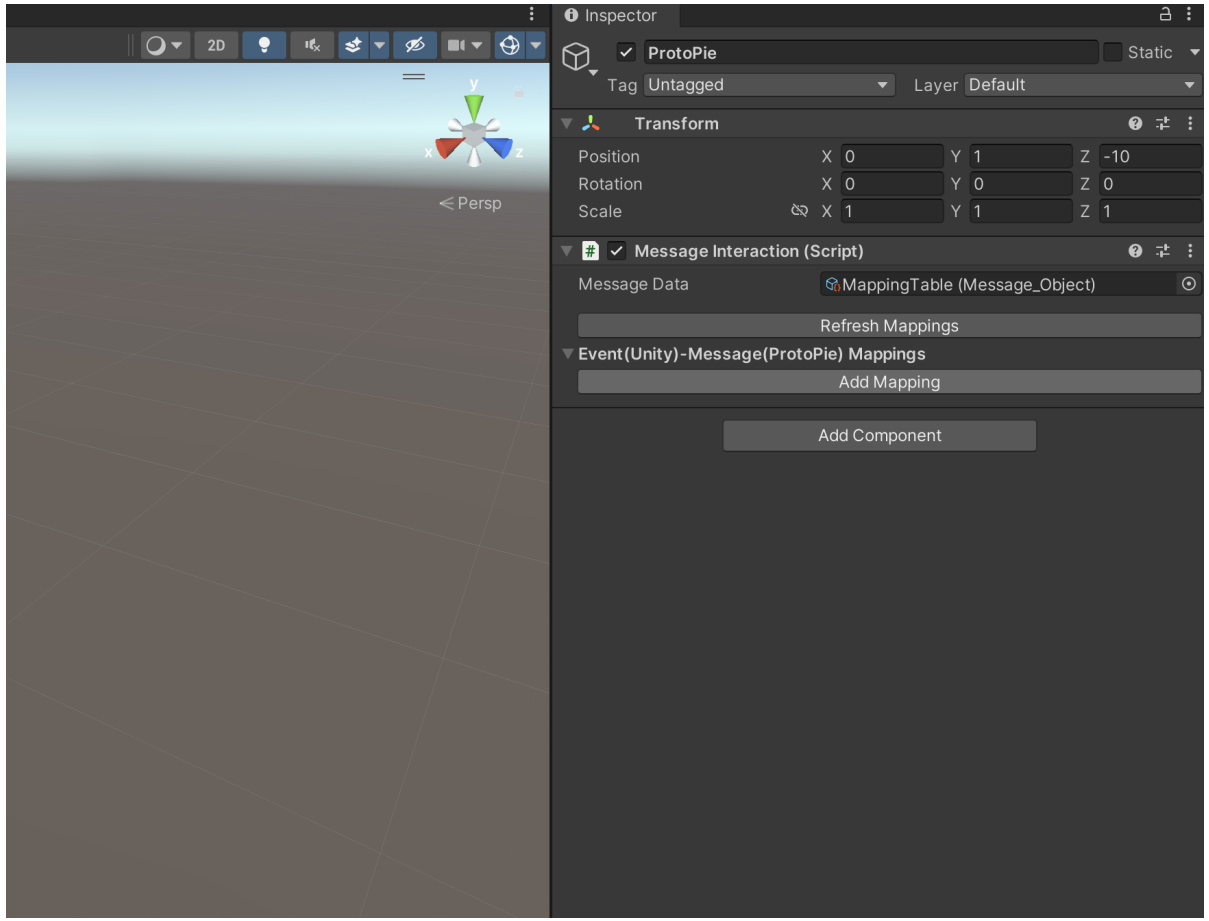
2. Add “MessageInteraction.cs” to ProtoPie object

Add “MessageInteraction.cs” (from ProtoPieUnityPackage) to ProtoPie object just created



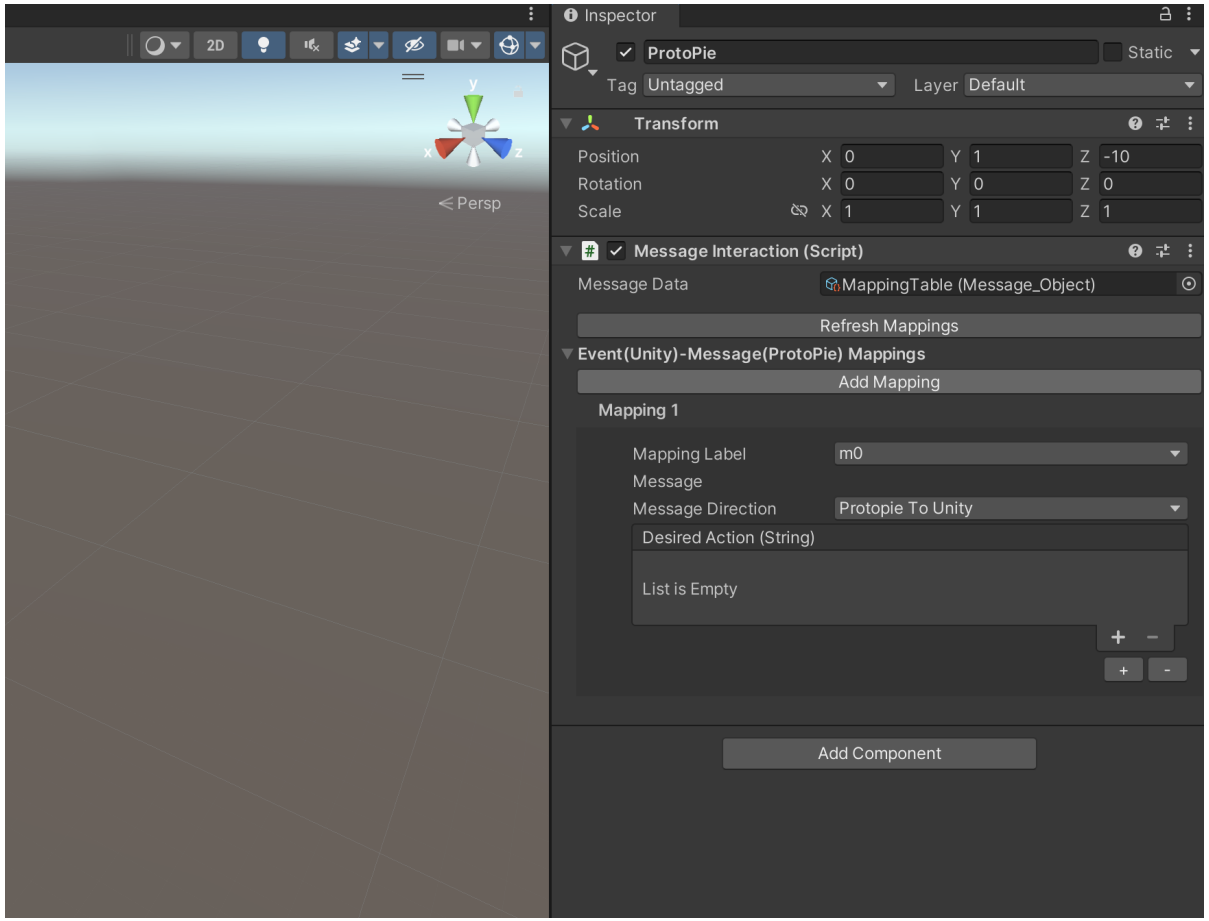
3. In Message Data field, select MappingTable

Add 'MappingTable.asset' to the Message Interaction component: 1) click the circle button next to Message Data, 2) click Scene, then 3) select MappingTable from the list. This configuration file allows you to specify the label, the exact message to be transmitted, and the message flow direction. The table is in YAML format for easy import into different platforms.

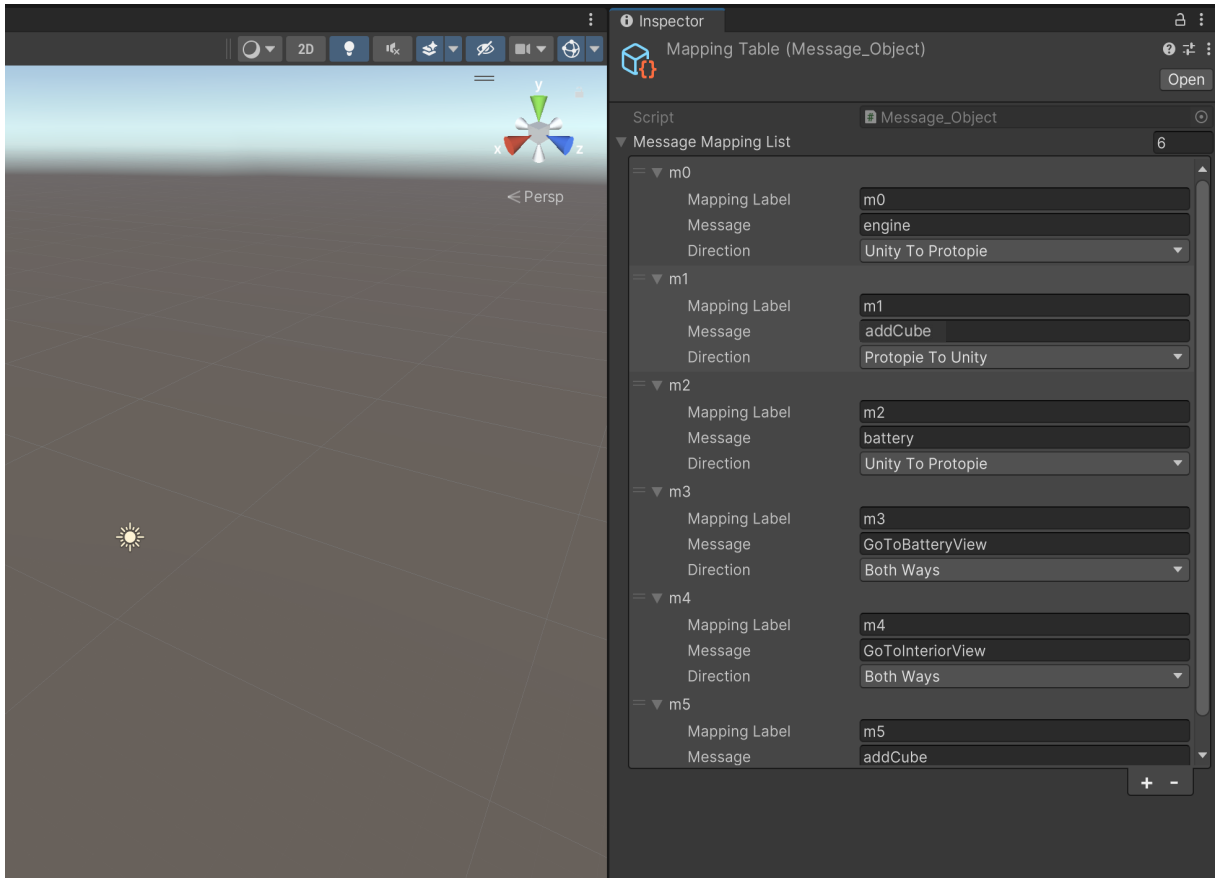


4. Press [Add Mapping] button

Press [Add Mapping] button to add the first mapping



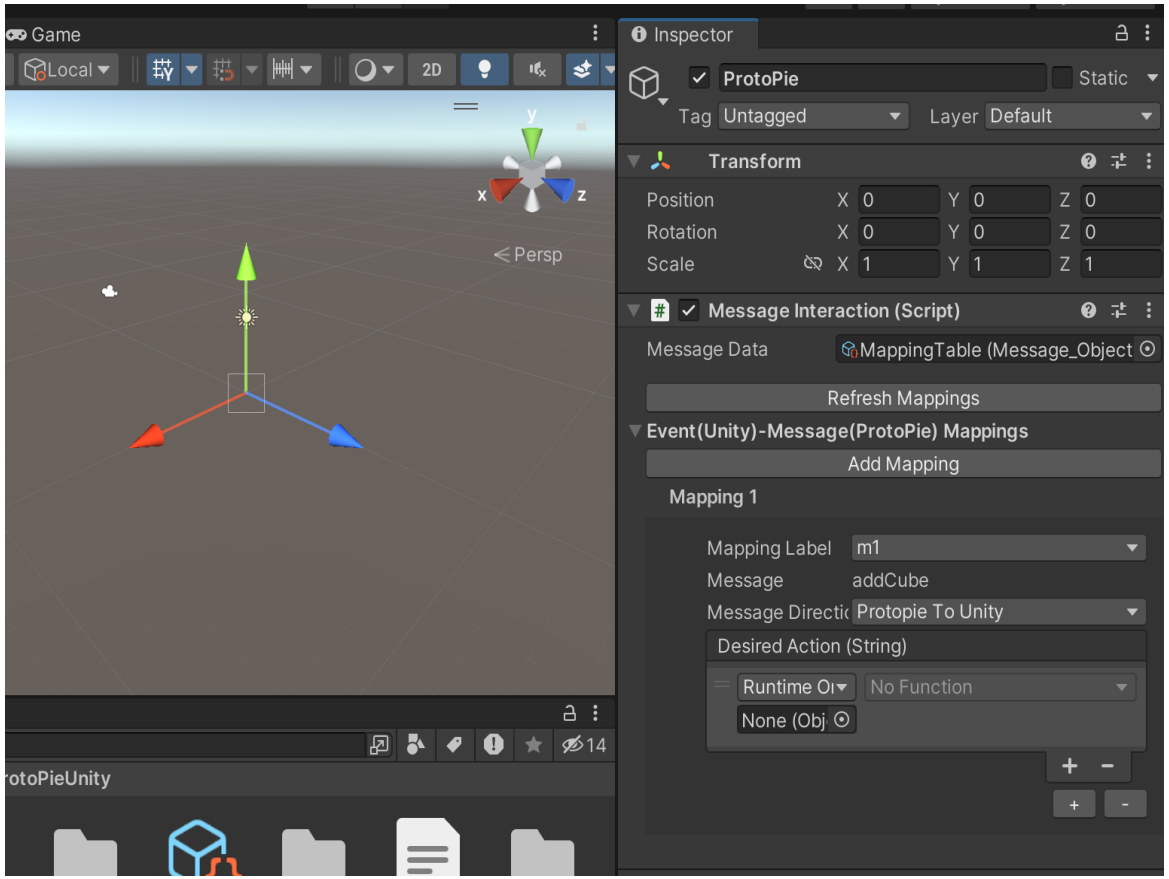
5. The first mapping added



6. Edit the message mapping list

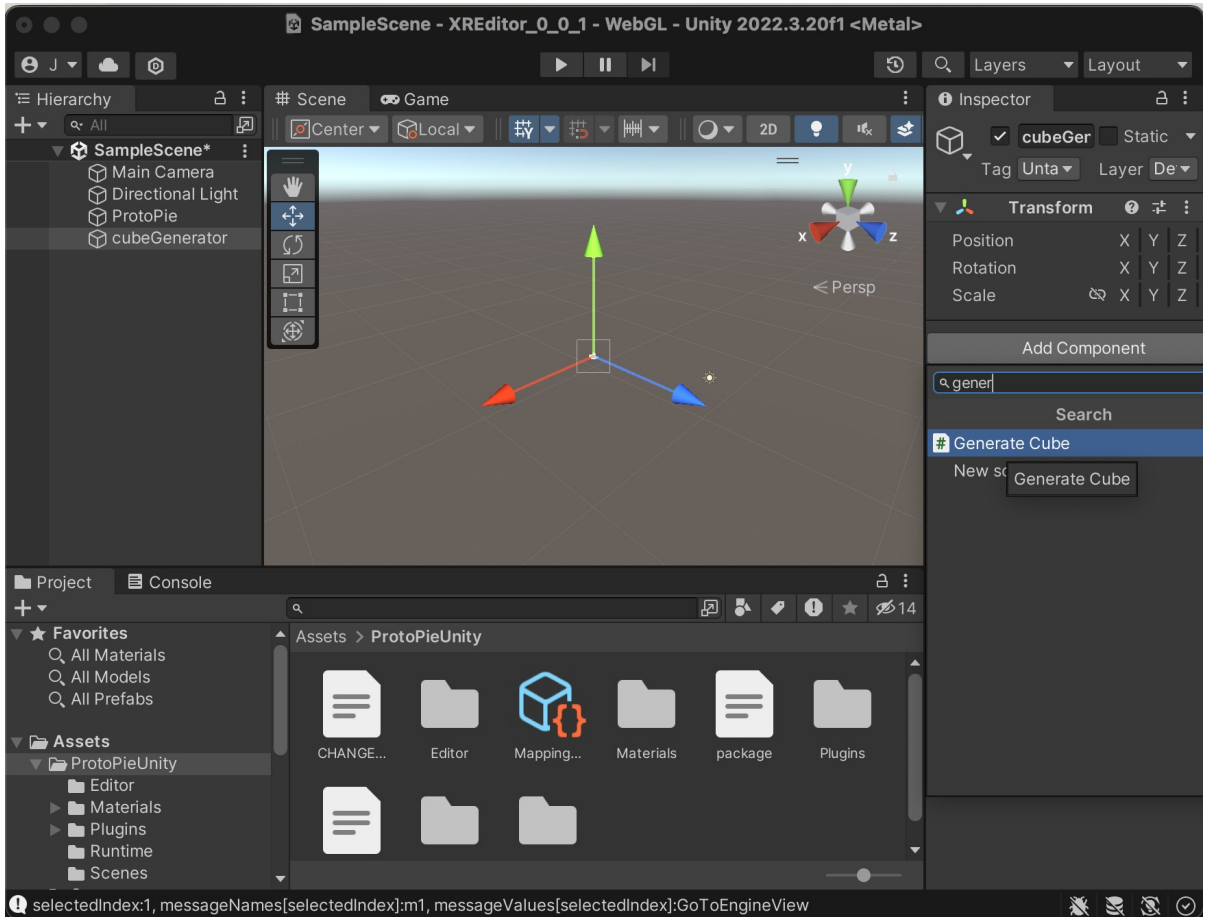
You can locate the mapping table in the package folder (MappingTable.asset), in which you can add/remove/edit the entries in the message mapping list. You can add a new message mapping or use the existing one. The second mapping “m1” will be used in the following example.

2. Add a mapping for a message from ProtoPie to Unity



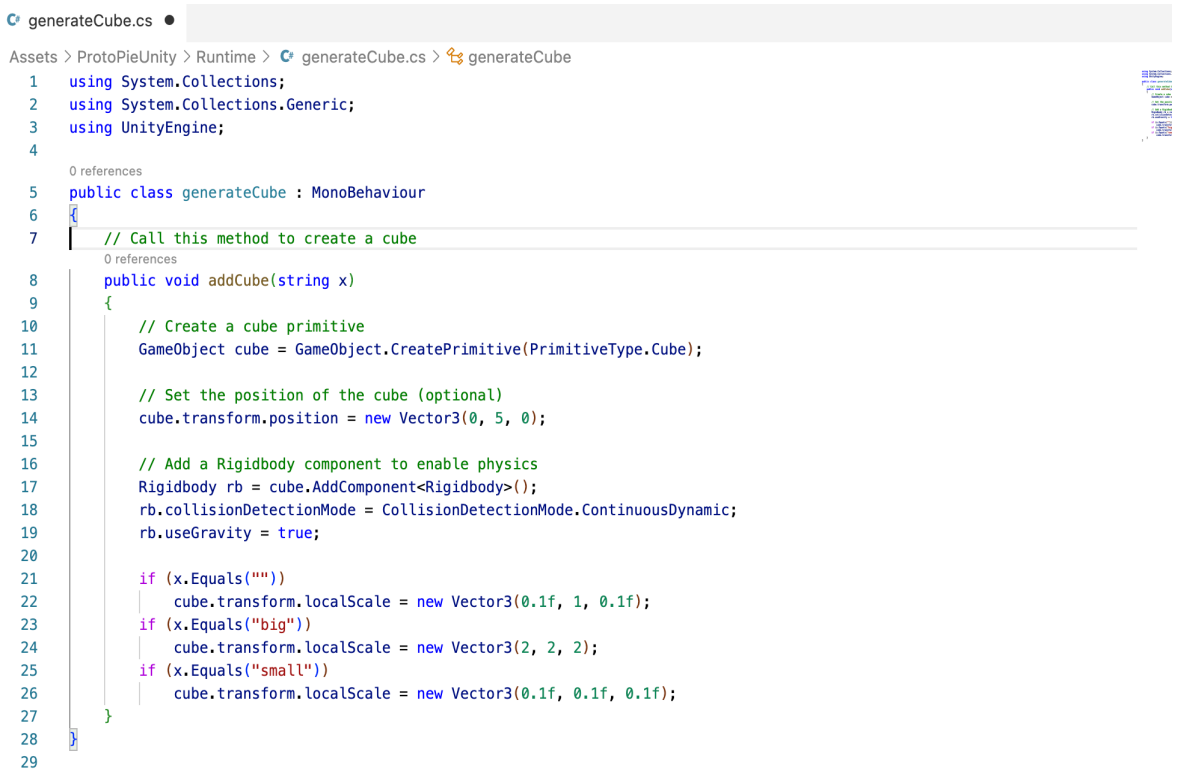
1. Select the message mapping using the mapping Label

Clicking the dropdown in the **Mapping Label** field will populate it with all the entries from the mapping message list. For example, selecting 'm1' will automatically complete the **Message** and **Message Direction** fields with the corresponding information. Additionally, based on the **Message Direction** selected—specifically 'ProtoPie To Unity' in this case—a **Desired Action** field will appear, allowing you to select the appropriate action (method/function) that should be executed.



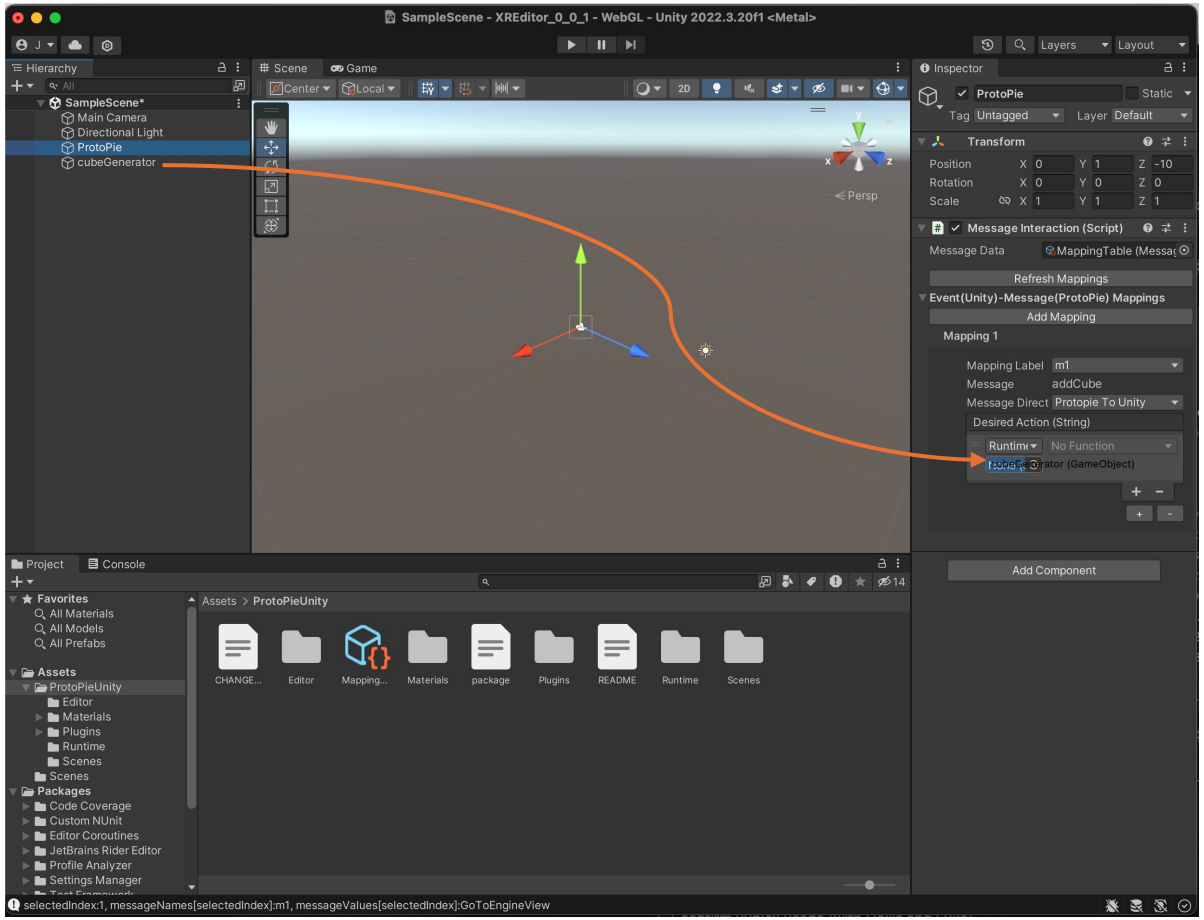
2. Choose an action for the message (1/5)

To select a method as the desired action, you must first determine its source object. For instance, in this case, the 'generateCube.cs' script (located in 'ProtoPieUnity/Runtime') has been attached to an object named 'cubeGenerator'.



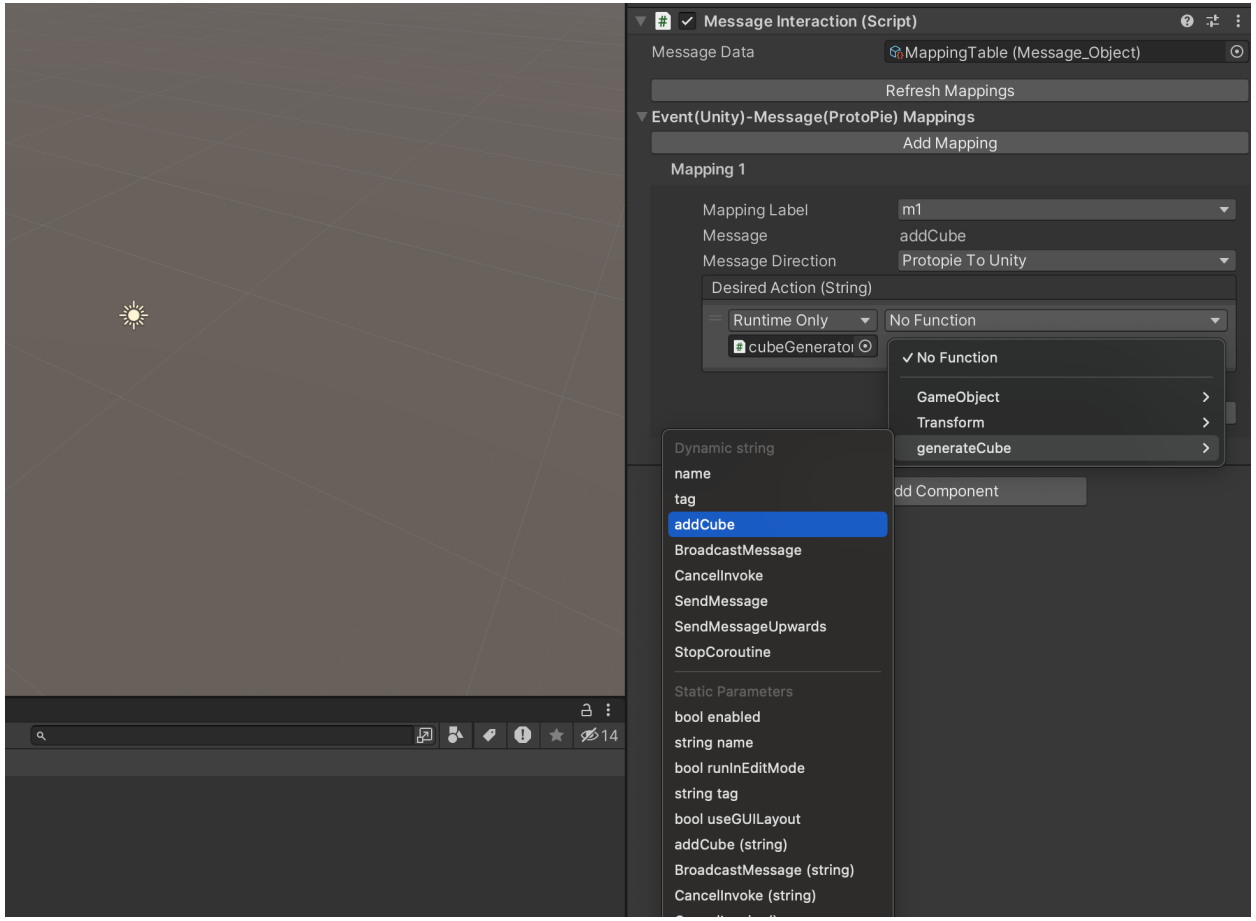
3. Choose an action for the message (2/5)

The 'cubeGenerator.cs' script includes a method named 'addCube(string)' that is responsible for creating a 3D cube within the Unity environment. This method accepts a string parameter, enabling the demonstration of varied behaviors based on the input provided. In this context, the argument passed will specify the cube's size. The 'addCube' method will be designated as the **Desired Action** in the forthcoming steps.



4. Choose an action for the message (3/5)

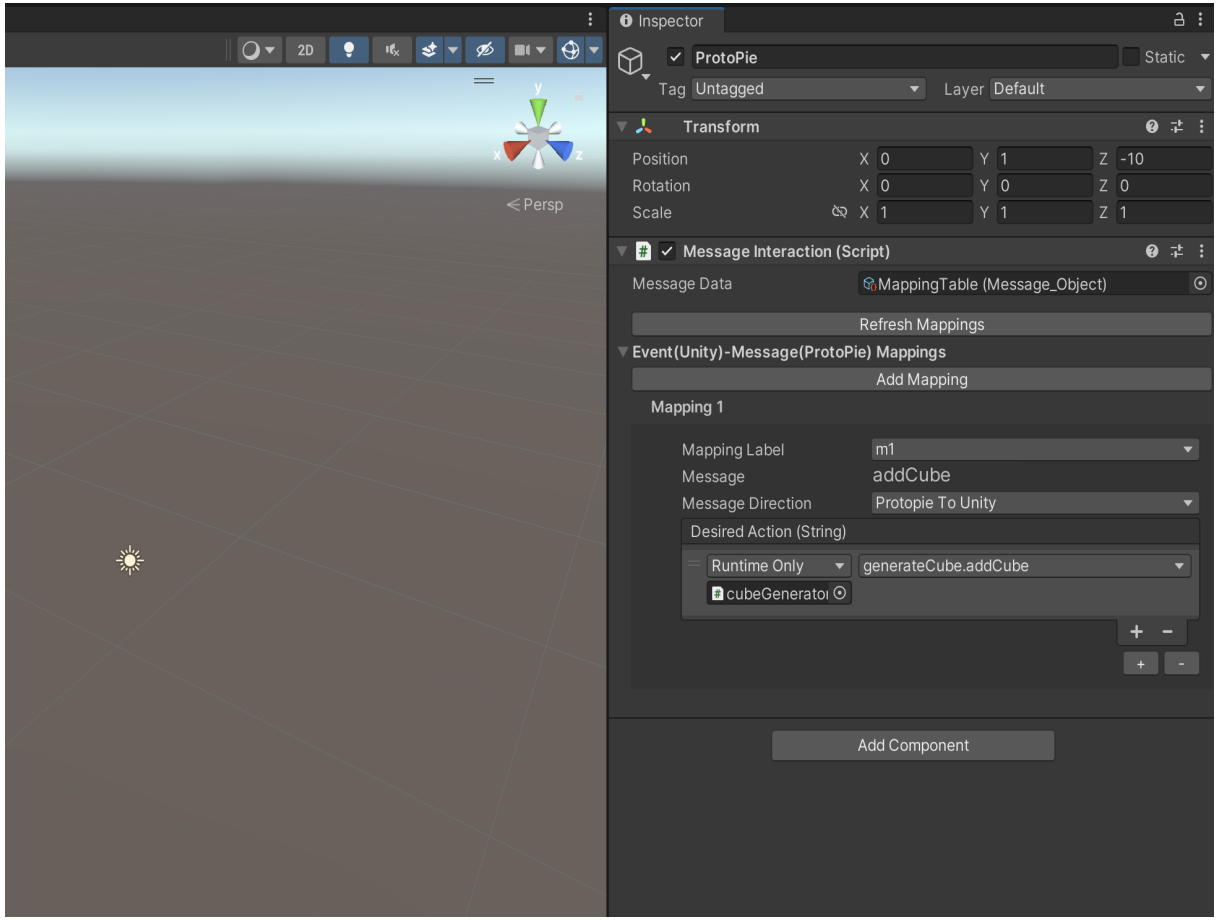
Drag&Drop the ‘cubeGenerator’ object to the **Desired Action** as shown in the figure.



5. Choose an action for the message (4/5)

From the **Desired Action** dropdown menu, select the **addCube** method listed under **Dynamic string**. This action will be invoked when the 'addCube' message is received from ProtoPie to Unity.

* Note: The 'addCube(string)' option under 'Static Parameters' is not configurable at runtime. Instead, the string value must be predefined within the Unity Editor prior to building the project.

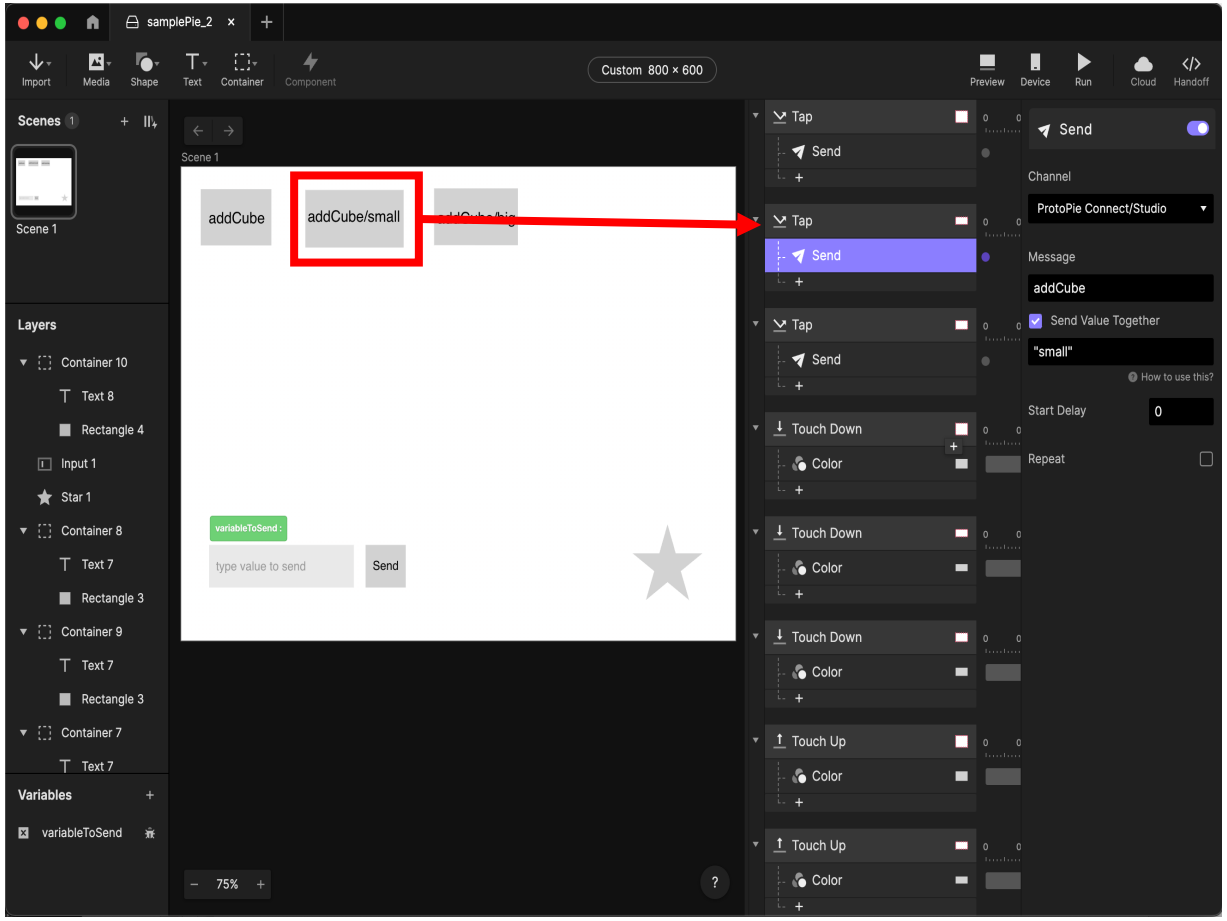


6. Choose an action for the message (5/5)

addCube method in generateCube.cs is now linked to the “addCube” message sent from ProtoPie. In other words, a cube will be added in the scene when the message “addCube” was sent from a pie.

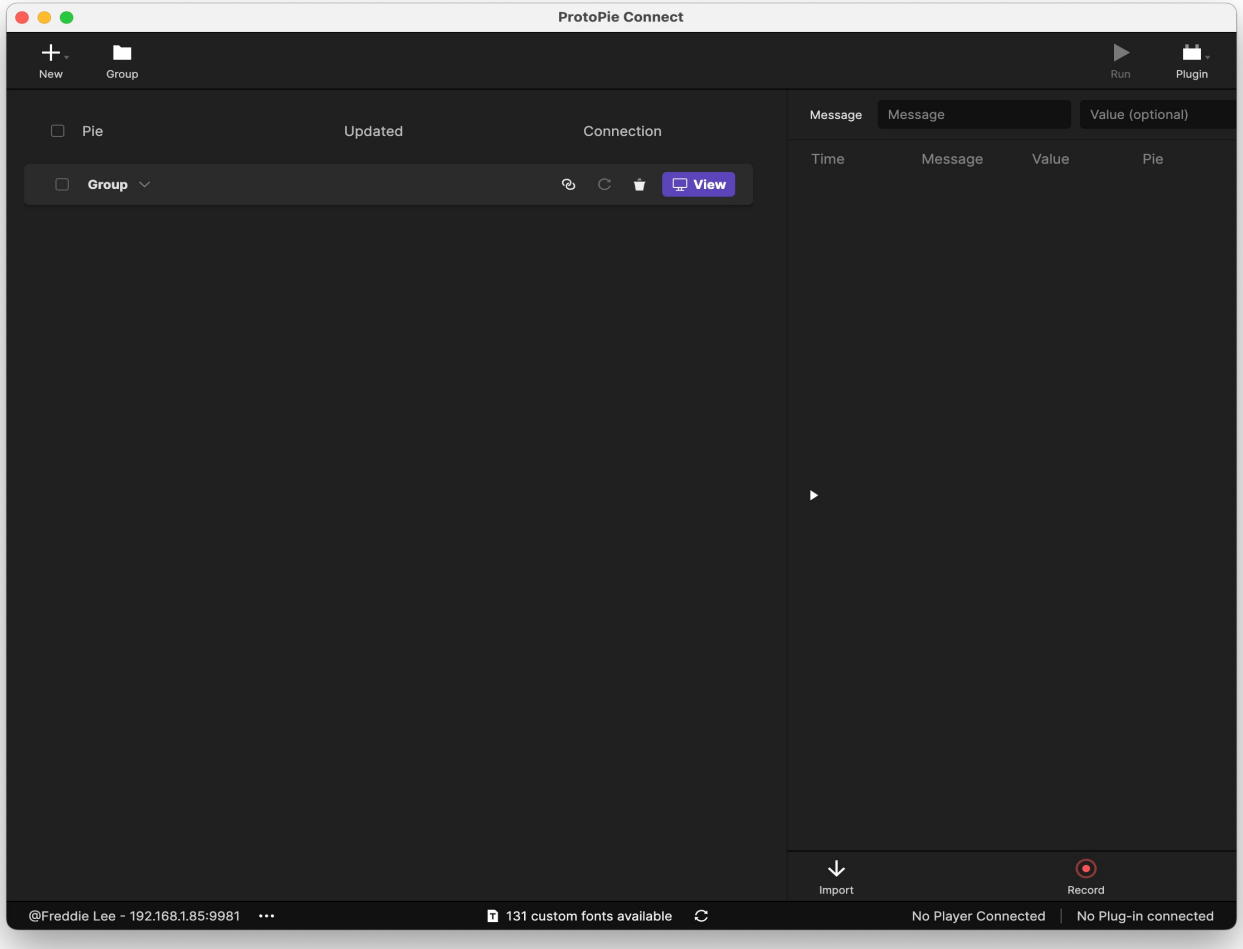
After linking the method, proceed to export the Unity project as a WebGL application.

* Before building, make sure to disable the Compression in Player Setting > Player > Publishing Settings > Compression Format



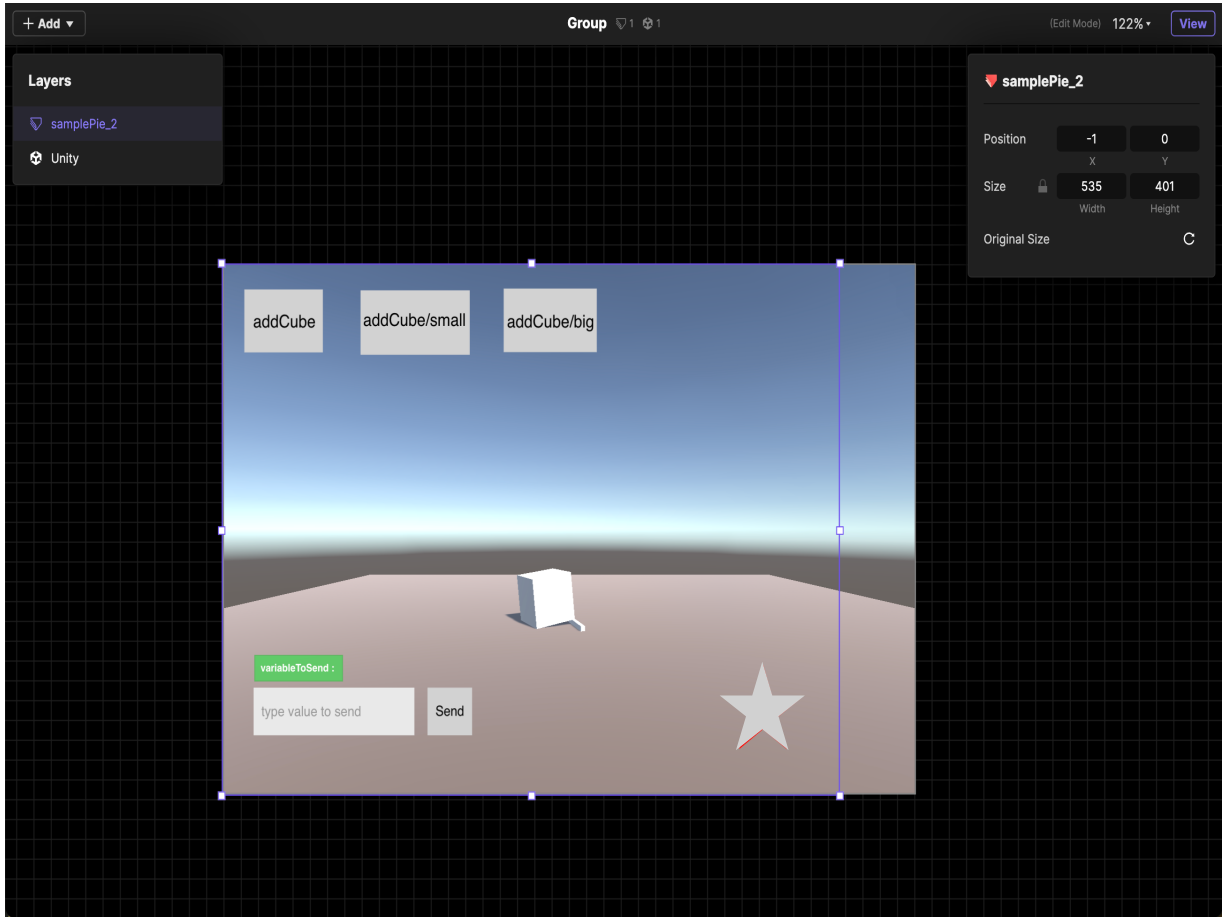
7. Design a pie to send the message

In ProtoPie studio, add the Send response and enter the message for the m1 mapping, which was “addCube” in this example, in the **Message** field. Along with the message, you can specify the size of the cube by sending a string input in the **Value** field (check the **Send Value Together** option). For instance, when you tap the second button from the top, it will trigger the "addCube" message accompanied by the string "small," indicating that a cube of small size should be generated in the Unity scene.



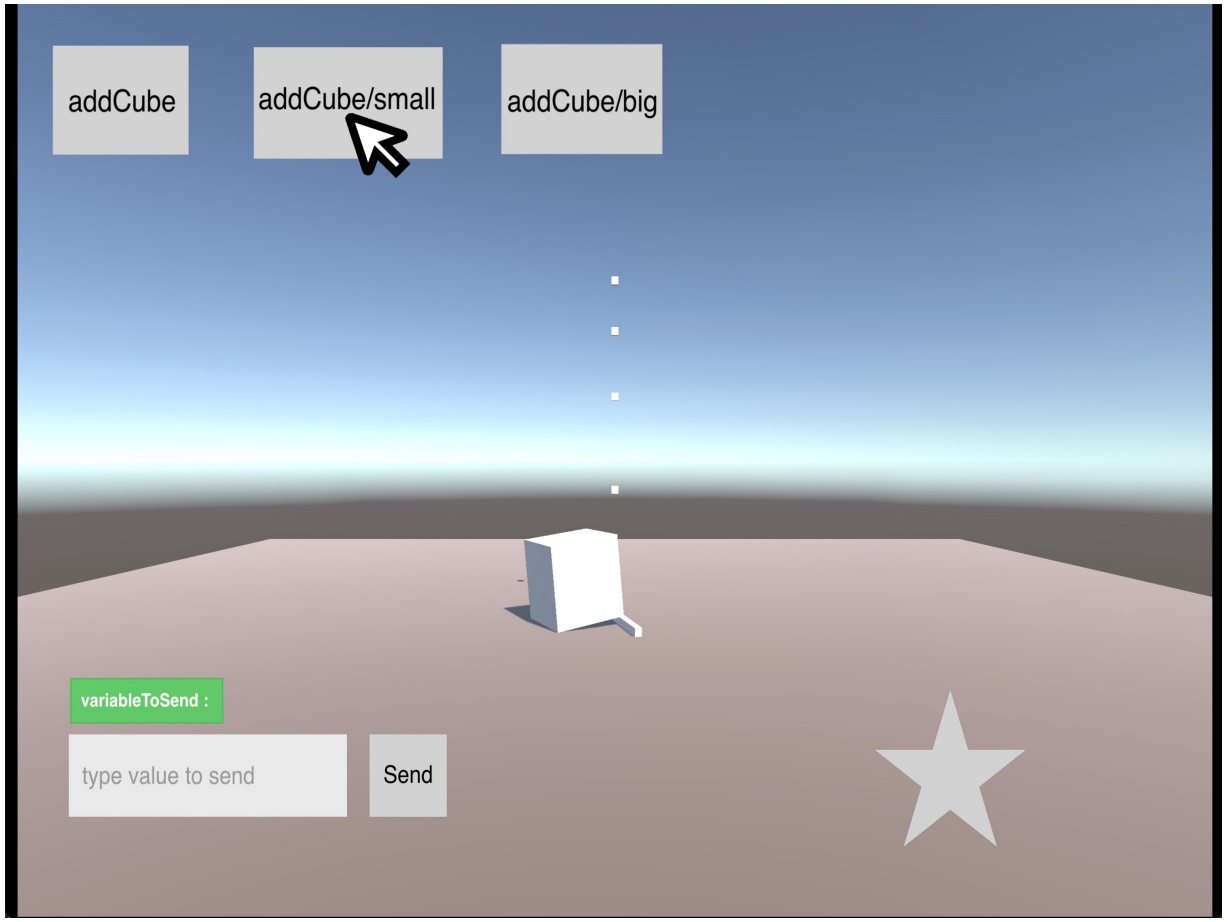
8.Create a Group and open in a web browser

In ProtoPie Connect, create a Group and click **View** button to open the empty canvas in a web browser.



9. Add pie and Unity layers

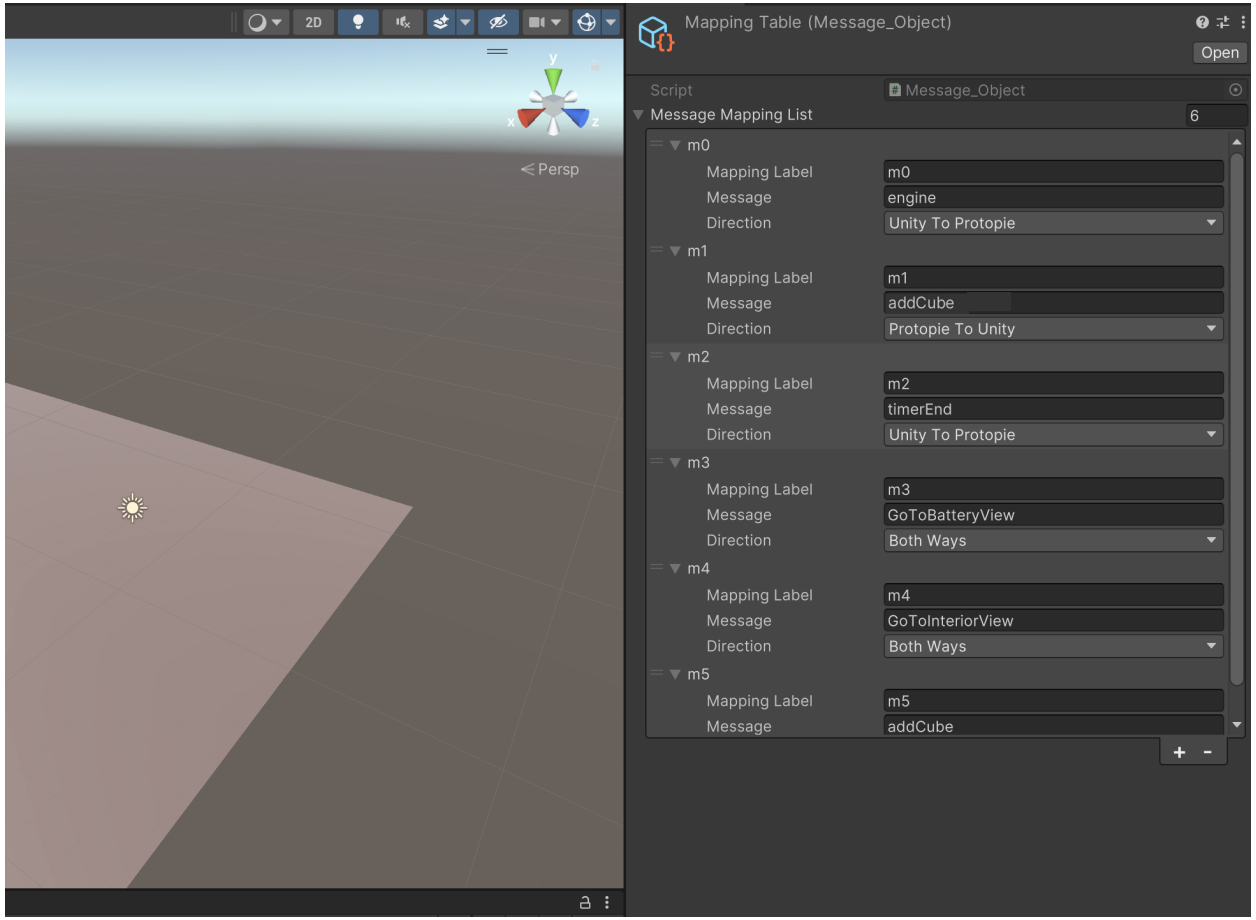
Right click on the empty canvas to open the UI for editing, add pie and Unity layers using the pie file and Unity WebGL build created in the previous steps.



10. Test the pie→Unity messaging

Try tapping the button that sends “addCube” message and see if the Unity layer generates small cubes in the scene.

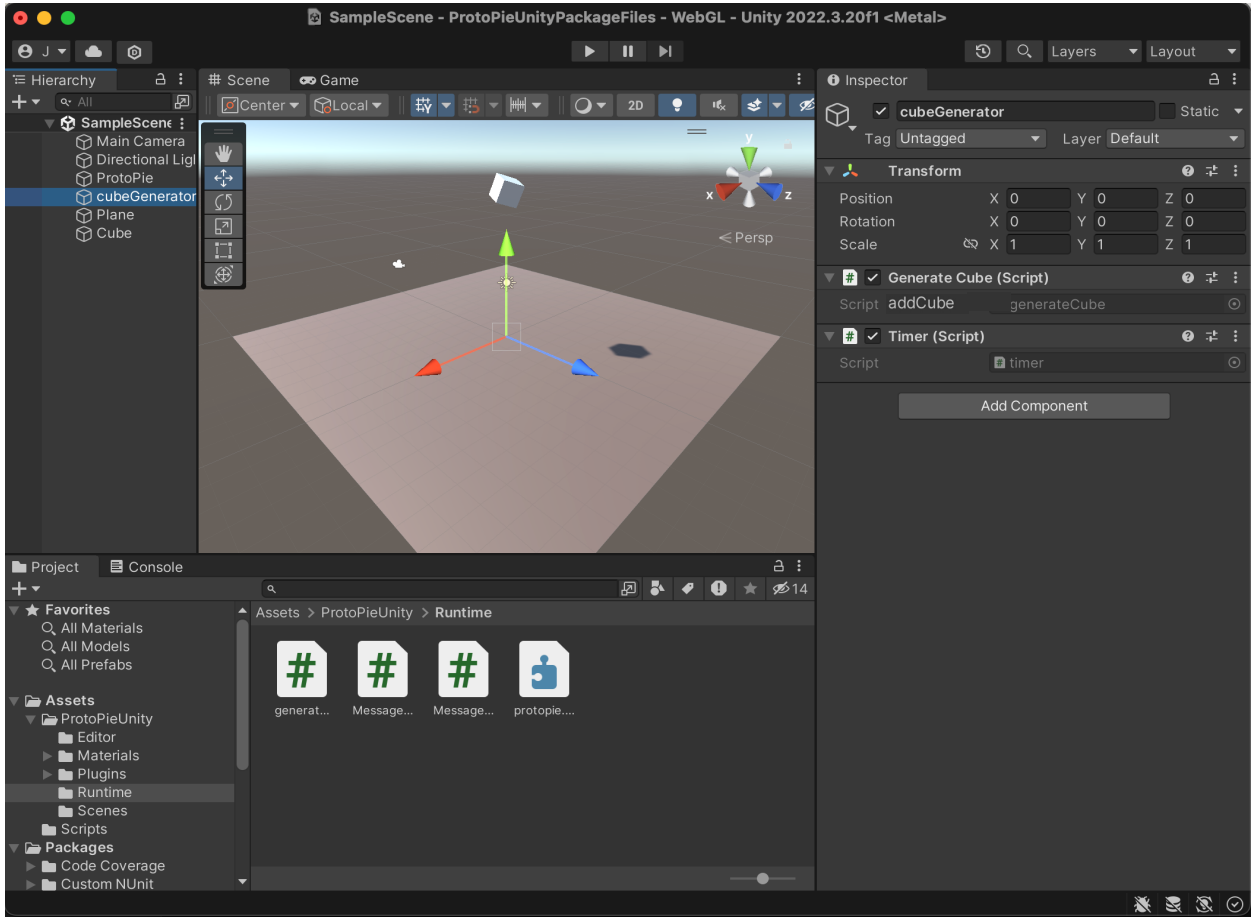
3. Add a mapping for a message from Unity to ProtoPie



1. Edit/add a new message mapping

In the mapping table, add a new entry or edit the existing one, as **m2** shown in the figure.

Message: timerEnd
Direction: Unity To ProtoPie

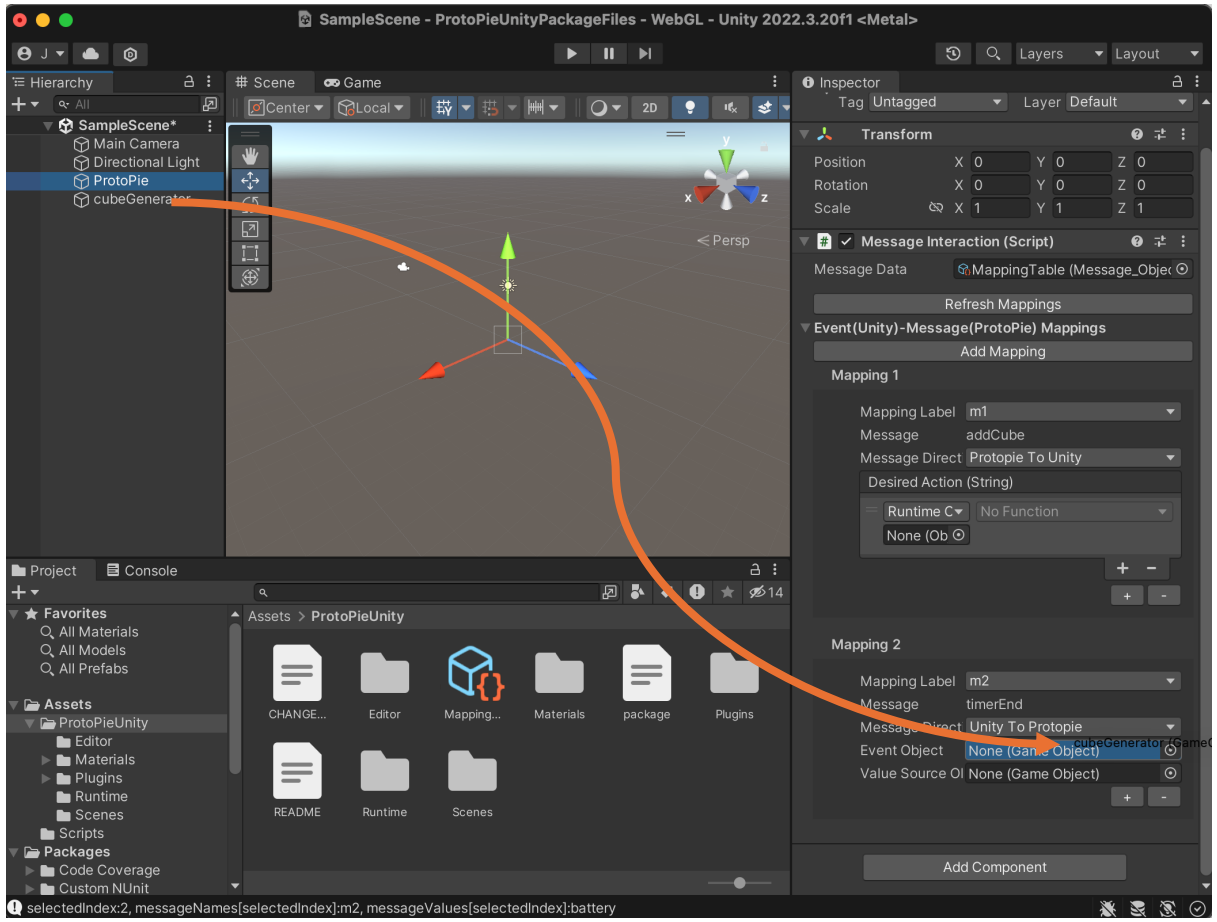


2. Create a script for event invoking

Attach a new script to the ‘cubeGenerator’ object. For example, the 'Timer.cs' script is added to provide the object with timing functionality.

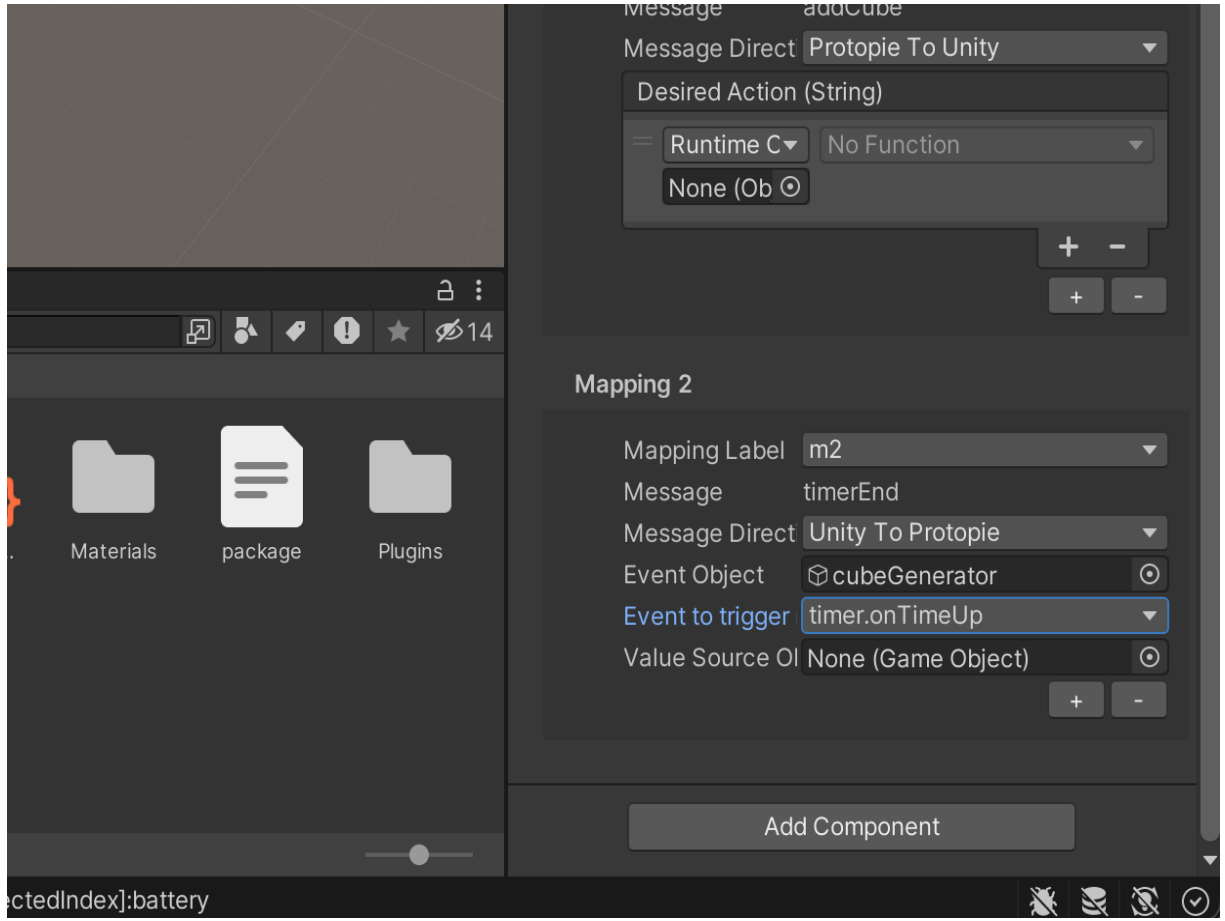
3. Define a UnityEvent and invoke it

In the script just created, define a UnityEvent. Trigger this event at the specific point in the script where the communication from Unity to ProtoPie should occur. In this example, onTimeUp event is triggered once the timer completes its countdown. You can make the UnityEvent public if you want to trigger it in any part of your Unity project.



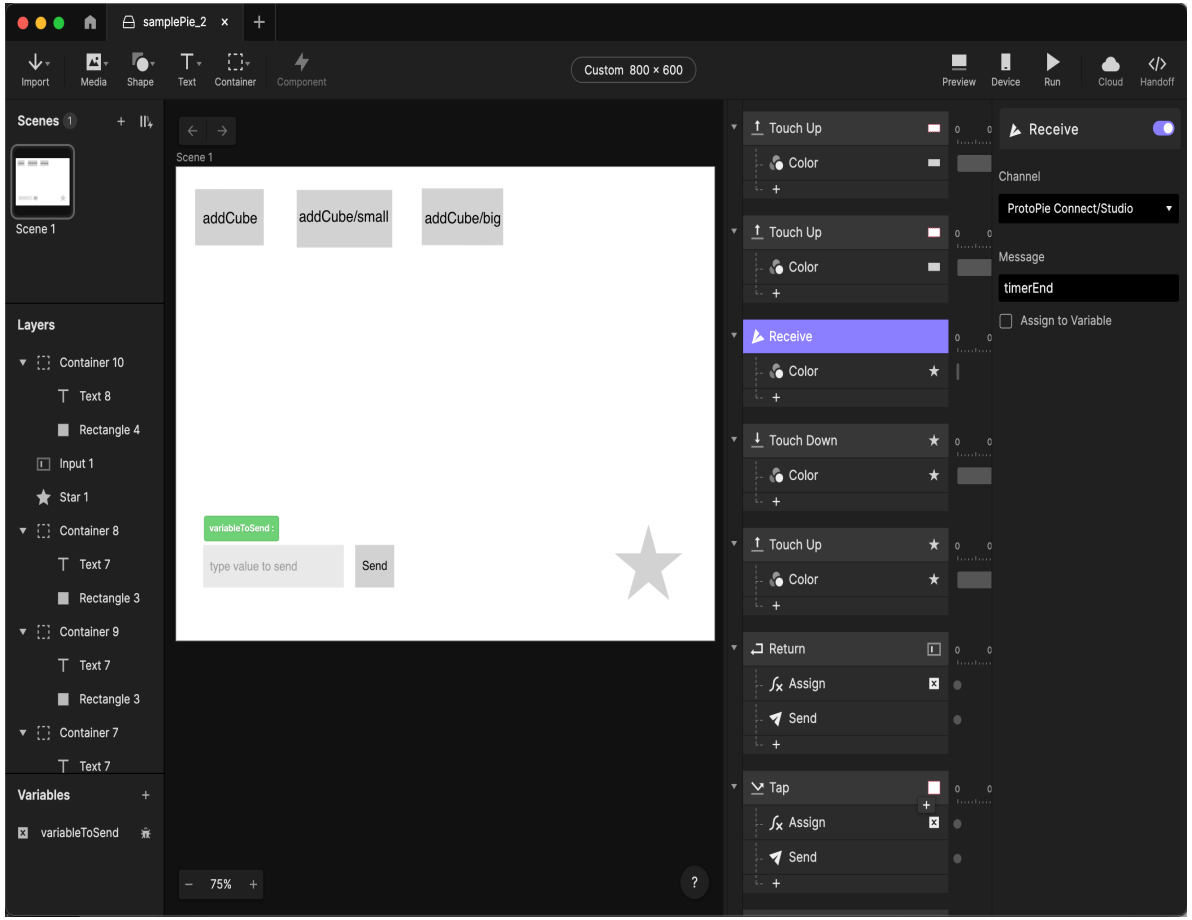
4. Add a new mapping for m2

In the Message Interaction component in ProtoPie object, add a new mapping and choose **m2**.
Drag&drop cubeGenerator object to **Event Object** field, as shown in this figure.



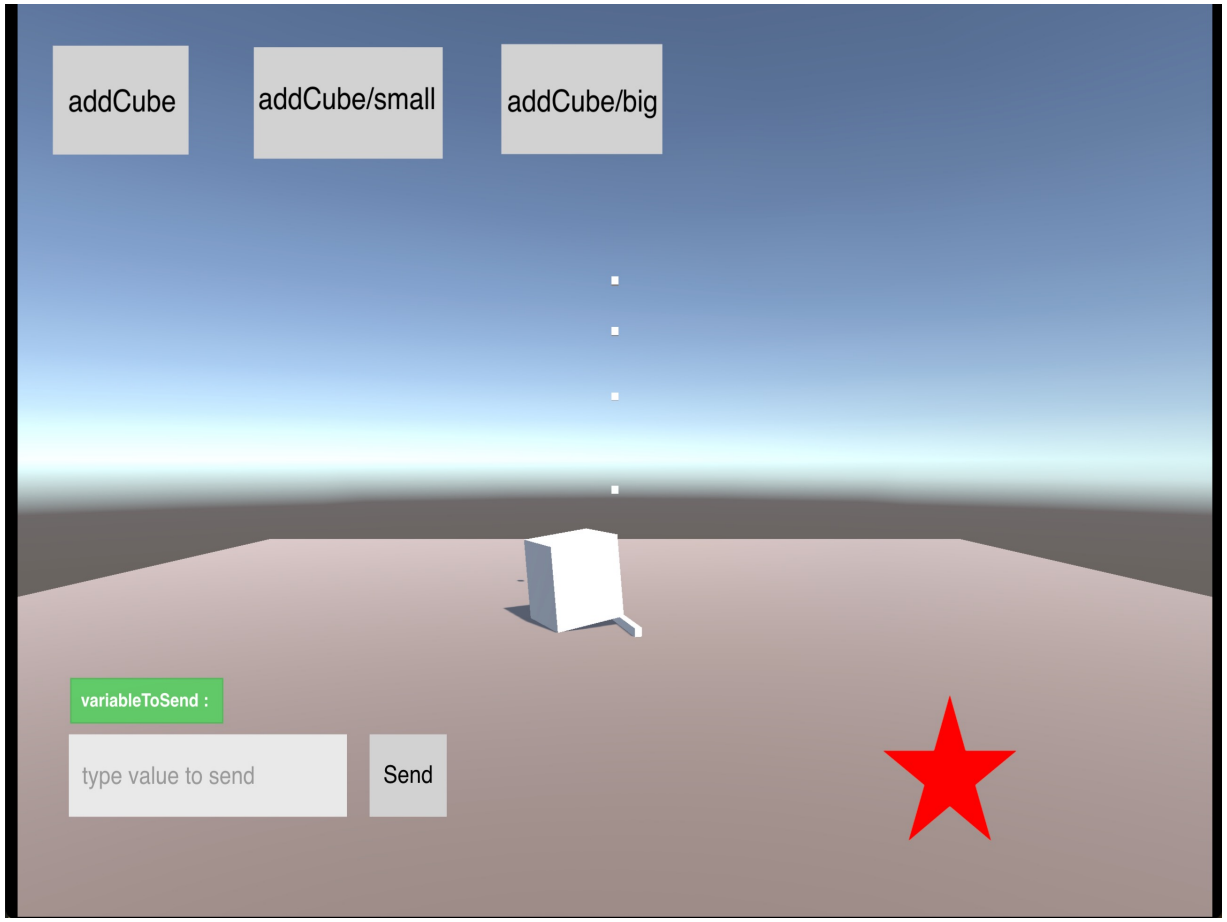
5. Choose an event to send message

Choose the 'onTimeUp' event to configure Unity to send a message to ProtoPie when the event fires. To transmit additional data with the message, drag an object into the **Value Source Object** field and select a public string variable from the script that is attached to that object.



6. Add Receive trigger in the pie

Add **Receive** trigger in the pie which is to be added as a pie layer (as in step 2.9) in ProtoPie Connect. In this example, the fill color of the star (in the right bottom) turns red when timerEnd message is sent from Unity to ProtoPie.



7. Test the UnityWebGL→pie messaging

See if the star turns red after the timer completes its countdown.